ANALYTICAL PROBLEMS

Q1. Histogram Equalization

1. Introduction and theory

In digital image processing, a grayscale image represents each pixel using an intensity value. In an 8-bit grayscale image, the intensity ranges from 0 to 255.

- 0 represents black.
- 255 represents white.
- Values between 0 and 255 represent different shades of gray.

Histogram: A histogram is a graphical representation of how frequently each intensity value occurs in an image.

Histogram equalization: It is an image enhancement technique that redistributes pixel intensities to improve the contrast of an image. It uses the cumulative distribution function (CDF) to map old intensities to new intensities.

Given problem

The grayscale image contains the following eight pixel values:

10, 10, 20, 20, 20, 30, 30, 40

We must:

1. Calculate the frequency of each intensity.
2. Construct the histogram.
3. Calculate the probability of each intensity.
4. Calculate the cumulative distribution function.
5. Apply histogram equalization.
6. Compare contrast before and after equalization.

Step 1: Identify the total number of pixels

Count the given values one by one.

Position	Pixel intensity
1	10
2	10
3	20
4	20
5	20
6	30
7	30
8	40

Therefore, the total number of pixels is

$N=8$

We use $N=8$ as the denominator when calculating probabilities.

Step 2: Find the frequency of each intensity

Frequency means the number of times an intensity occurs.

For intensity 10:

The value 10 occurs twice.

$n(10)=2$

For intensity 20:

The value 20 occurs three times.

$n(20)=3$

For intensity 30:

The value 30 occurs twice.

$n(30)=2$

For intensity 40:

The value 40 occurs once.

$n(40)=1$

Check the total frequency:

2+3+2+1=8

The total equals the number of pixels, so our frequency calculation is correct.

Step 3: Construct the histogram table

Intensity r	Frequency n(r)
10	2
20	3
30	2
40	1
Total	8

The histogram has intensity on the horizontal axis and frequency on the vertical axis.

Histogram representation

10

2 pixels

20

3 pixels

30

2 pixels

40

1 pixels

Only the four distinct intensity values are shown.

Notice that intensity 20 has the tallest bar because it occurs three times. Intensity 40 has the shortest bar because it occurs only once.

Step 4: Calculate the probability of each intensity

The probability mass function (PMF) tells us the proportion of pixels having a particular intensity.

Formula:

$P(r_k)=(n_k) \div (N)$

Where:

- P(r_k) = probability of intensity r_k.
- n_k = frequency of that intensity.
- N = total number of pixels, which is 8.

Now calculate each probability separately.

For intensity 10:

Frequency = 2, total pixels = 8.

P(10)=(2) ÷ (8)

P(10)=0.25

For intensity 20:

Frequency = 3, total pixels = 8.

P(20)=(3) ÷ (8)

P(20)=0.375

For intensity 30:

Frequency = 2, total pixels = 8.

P(30)=(2) ÷ (8)

P(30)=0.25

For intensity 40:

Frequency = 1, total pixels = 8.

P(40)=(1) ÷ (8)

P(40)=0.125

Check the probabilities:

0.25+0.375+0.25+0.125=1

The sum is 1, as expected.

Step 5: Calculate the cumulative distribution function (CDF)

The CDF is the cumulative sum of the probabilities, meaning we keep adding the probabilities from the smallest intensity upwards.

Formula:

$CDF(r_k)=\Sigma_{j=0}^{k}P(r_j)$

Let's calculate each value without skipping any addition.

For intensity 10:

This is the first intensity, so its CDF equals its own probability.

$CDF(10)=P(10)$

boxed$CDF(10)=0.25$

For intensity 20:

Add the probability of intensity 10 and the probability of intensity 20.

$CDF(20)=P(10)+P(20)$

$=0.25+0.375$

boxed$CDF(20)=0.625$

For intensity 30:

Add the previous CDF and the probability of intensity 30.

$CDF(30)=CDF(20)+P(30)$

$=0.625+0.25$

boxed$CDF(30)=0.875$

For intensity 40:

Add the previous CDF and the probability of intensity 40.

$CDF(40)=CDF(30)+P(40)$

$=0.875+0.125$

boxed$CDF(40)=1.00$

Why does the final CDF equal 1? Because we have included every pixel in the image.

Step 6: Apply the histogram equalization formula

The question gives the formula:

$s=\text{round}((L-1)\times \text{CDF})$

Here:

- s = new intensity.
- L = number of possible intensity levels.
- CDF = cumulative probability.
- operatornameround = round to the nearest integer.

The image uses 8-bit grayscale, so:

$L=256$

Therefore:

$L-1=256-1=255$

Our formula becomes:

$$\boxed{s=\text{round}(255\times \text{CDF})}$$

Now calculate the new intensity for each original intensity.

For intensity 10:

From Step 5:

$\text{CDF}(10)=0.25$

Substitute into the formula:

$s(10)=\text{round}(255\times 0.25)$

Multiply:

$255\times 0.25=63.75$

Round 63.75 to the nearest integer:

$$\boxed{s(10)=64}$$

Thus, every original pixel having intensity 10 becomes 64.

For intensity 20:

CDF(20)=0.625

Substitute:

s(20)=round(255×0.625)

Multiply:

255×0.625=159.375

Round to the nearest integer:

boxeds(20)=159

Thus, every original pixel having intensity 20 becomes 159.

For intensity 30:

CDF(30)=0.875

Substitute:

s(30)=round(255×0.875)

Multiply:

255×0.875=223.125

Round to the nearest integer:

boxeds(30)=223

Thus, every original pixel having intensity 30 becomes 223.

For intensity 40:

CDF(40)=1

Substitute:

s(40)=round(255×1)

s(40)=round(255)

boxeds(40)=255

Step 7: Prepare the final equalization table

Original intensity	Frequency	CDF	Calculation	New intensity
10	2	0.25	round(255 × 0.25)	64
20	3	0.625	round(255 × 0.625)	159
30	2	0.875	round(255 × 0.875)	223
40	1	1.00	round(255 × 1.00)	255

Step 8: Construct the new image

Original pixel sequence:

[10,10,20,20,20,30,30,40]

Replace each intensity using the table:

- First 10 → 64.
- Second 10 → 64.
- First 20 → 159.
- Second 20 → 159.
- Third 20 → 159.
- First 30 → 223.
- Second 30 → 223.
- 40 → 255.

Therefore, the equalized image is:

boxed[64,64,159,159,159,223,223,255]

Step 9: Compare contrast before and after equalization

Before equalization:

I_min=10, I_max=40

The intensity range is:

40-10=30

After equalization:

I_min=64, I_max=255

The intensity range is:

255-64=191

Property	Before	After
Minimum intensity	10	64
Maximum intensity	40	255
Intensity range	30	191

The intensity range has increased from 30 to 191. This indicates a wider spread of intensity values, which can improve contrast.

Important theory: Histogram equalization does not guarantee a perfectly uniform histogram in a small discrete image. It remaps intensities based on the CDF, and several original intensity levels can map to different new levels.

Exam conclusion

Histogram equalization enhances image contrast by transforming original pixel intensities using the cumulative distribution function. For the given image, the equalized sequence is [64,64,159,159,159,223,223,255], and the intensity range increases from 30 to 191.

Q2. Spatial Filtering — Averaging Filter and Median Filter

1. Introduction and theory

Spatial filtering is a technique used in digital image processing to modify an image by performing calculations directly on its pixels.

A filter, also called a mask or kernel, is a small matrix that moves across an image. At each position, the filter examines a group of neighbouring pixels and calculates a new value for the pixel being processed.

For example, a 3×3 filter contains nine values:

beginbmatrix a&b&c d&e&f g&h&i

The value e is the centre pixel. The remaining eight values are its neighbours.

There are two filtering methods in this question:

- Averaging filter: Calculates the arithmetic mean of the pixels in the neighbourhood.
- Median filter: Arranges the pixel values in ascending order and selects the middle value.

Both methods can reduce noise, but they behave differently around edges and extreme pixel values.

2. Given problem

The given 3×3 grayscale image is

I= 10&20&30 20&100&20 10&20&30

The centre pixel is 100.

We must calculate the new centre pixel using an averaging filter, calculate it again using a median filter, and compare both results.

Part A: Averaging filter

Step 1: Understand the purpose of averaging

An averaging filter replaces a pixel with the average of its surrounding pixels, including the pixel itself.

It is commonly used for:

- Smoothing an image.
- Reducing small intensity variations.
- Reducing certain types of noise.
- Removing fine details along with noise when the smoothing is strong.

The standard 3×3 averaging kernel is

K=(1) ÷ (9) 1&1&1 1&1&1 1&1&1

Why is every value 1? Because every pixel contributes equally to the sum.

Why do we divide by 9? Because there are nine pixels in a 3×3 neighbourhood.

Step 2: Write the original neighbourhood

beginbmatrix 10&20&30 20&100&20 10&20&30

List every value individually:

7. Top-left pixel = 10
8. Top-middle pixel = 20
9. Top-right pixel = 30
10. Middle-left pixel = 20
11. Centre pixel = 100
12. Middle-right pixel = 20
13. Bottom-left pixel = 10
14. Bottom-middle pixel = 20
15. Bottom-right pixel = 30

This list ensures that no pixel is accidentally omitted.

Step 3: Write the averaging formula

textNew centre value =fractextSum of all pixelstextNumber of pixels

Since there are nine pixels,

textNew centre value=(S) ÷ (9)

where S is the sum of the nine intensity values.

Step 4: Calculate the sum row by row

First row:

10+20+30=60

Second row:

20+100+20=140

Third row:

10+20+30=60

Add the three row totals:

S=60+140+60

boxedS=260

Step 5: Divide the sum by nine

Substitute the sum into the formula:

textAverage=(260) ÷ (9)

Perform the division:

260÷9=28.888ldots

Therefore, the exact average is

boxed28.888ldots

If the result must be represented as an integer grayscale intensity, round to the nearest whole number:

boxedtextNew centre pixel=29

Notice that the original centre pixel was 100, but after averaging, it becomes approximately 29. The large value has been pulled towards the smaller neighbouring values.

Step 6: Replace only the centre pixel

The question specifically says to keep all other pixels unchanged.

Original image:

beginbmatrix 10&20&30 20&100&20 10&20&30

Replace the centre value 100 with 29:

boxed I_textaverage= 10&20&30 20&29&20 10&20&30

The other eight pixels remain unchanged.

Part B: Median filter

Step 1: Understand the purpose of a median filter

A median filter is a nonlinear spatial filter. Instead of calculating the average, it sorts the neighbouring pixel intensities and selects the middle value.

It is particularly useful for removing salt-and-pepper noise, which appears as isolated very bright or very dark pixels.

For example, suppose the neighbourhood contains:

10, 10, 20, 20, 20, 20, 30, 30, 100

The value 100 is much larger than most of the other values. The median is less affected by such an extreme value than the mean.

Step 2: Extract all nine pixel values

From the original image,

beginbmatrix 10&20&30 20&100&20 10&20&30

The values are:

10, 20, 30, 20, 100, 20, 10, 20, 30

Step 3: Arrange the values in ascending order

Sorting means arranging values from the smallest to the largest.

- The smallest value is 10.
- The next value is 10.
- The next four values are 20.
- The next two values are 30.
- The largest value is 100.

The sorted sequence is

boxed10, 10, 20, 20, 20, 20, 30, 30, 100

Check the count: there are nine values, exactly as required.

Step 4: Find the middle position

For an odd number of observations, the median position is

textMedian position=(n+1) ÷ (2)

Here, n=9.

Substitute:

textMedian position=(9+1) ÷ (2)

=(10) ÷ (2)

boxedtextMedian position=5

Therefore, we must select the fifth value in the sorted list.

Step 5: Identify the fifth value

Write the sorted values with their positions.

Position	Value
1	10
2	10
3	20
4	20
5	20
6	20
7	30
8	30
9	100

The fifth value is 20.

Therefore,

boxedtextMedian=20

Step 6: Replace the centre pixel

Replace the original centre value 100 with 20, keeping all the other pixels unchanged.

boxed I_textmedian= 10&20&30 20&20&20 10&20&30

Part C: Compare the two filters

Property	Averaging filter	Median filter
Calculation	Sum divided by 9	Middle sorted value
Centre result	28.888… (approximately 29)	20
Filter type	Linear	Nonlinear
Extreme pixel influence	Can be significant	Usually limited
Main application	Smoothing and reducing variations	Removing impulse noise
Effect on edges	Can blur edges	Often preserves edges better

Why is the median better in this example?

The original centre value is 100, while all its neighbours are between 10 and 30.

The averaging filter calculates

frac2609=28.888ldots

The large centre value increases the average.

The median filter sorts all nine values and chooses 20. Because the median depends on the middle-ranked value rather than the total sum, the extreme value 100 does not pull the result upwards in the same way.

Exam conclusion: The averaging filter produces a centre intensity of approximately 29, whereas the median filter produces 20. For this neighbourhood, the median filter is generally preferable when the value 100 represents an isolated noise spike. However, the best filter depends on the type of noise and the image details that need to be preserved.

Q3. Laplacian Filter for Image Sharpening

1. Introduction and theory

Image sharpening is a digital image processing operation used to make edges and fine details more noticeable.

A blurred image may contain weak transitions between neighbouring pixel intensities. Sharpening emphasizes these transitions, helping boundaries and small details stand out.

The Laplacian filter is a second-order derivative filter. It measures how the intensity of a pixel differs from the intensities around it.

The Laplacian is commonly used for:

- Detecting rapid intensity changes.
- Enhancing edges.
- Sharpening images.
- Emphasizing fine image details.

Unlike a smoothing filter, which reduces local variations, the Laplacian highlights local intensity changes.

2. Given problem

The grayscale image is

I= 50&50&50 50&100&50 50&50&50

The Laplacian mask is

K= 0&-1&0 -1&4&-1 0&-1&0

We must calculate the Laplacian response at the centre pixel and add it to the original centre intensity.

Step 1: Identify the centre pixel

The centre element of the image is

boxedI_c=100

The four direct neighbours are:

- Top = 50
- Bottom = 50

- Left = 50
- Right = 50

The four corner pixels are also 50, but the corresponding corner positions in the Laplacian mask contain zero.

This is important: the corner pixels do not contribute to the response for this particular mask.

Step 2: Understand the mask

K= 0&-1&0 -1&4&-1 0&-1&0

Read the mask position by position:

Position	Image value	Mask value
Top-left	50	0
Top-middle	50	−1
Top-right	50	0
Middle-left	50	−1
Centre	100	4
Middle-right	50	−1
Bottom-left	50	0
Bottom-middle	50	−1
Bottom-right	50	0

The centre has a positive weight of 4. The four direct neighbours each have a weight of −1.

Step 3: Multiply corresponding values

The filter response is calculated by multiplying each image value by the mask value in the same position.

Top row:

(0×50)+(-1×50)+(0×50)

Calculate each product:

0+(-50)+0

boxedtextTop-row contribution=-50

Middle row:

$(-1\times50)+(4\times100)+(-1\times50)$

Calculate each product:

$-50+400-50$

boxedtextMiddle-row contribution=300

Bottom row:

$(0\times50)+(-1\times50)+(0\times50)$

Calculate:

$0-50+0$

boxedtextBottom-row contribution=-50

Step 4: Add all the contributions

The total Laplacian response is

$R=(-50)+300+(-50)$

First add the negative contributions:

$-50-50=-100$

Then add the positive contribution:

$R=300-100$

Therefore,

boxedR=200

An equivalent, shorter calculation is

$R=4(100)-50-50-50-50$

$=400-200$

boxedR=200

Both methods give the same answer.

Step 5: Understand the meaning of the response

The centre pixel is 100, while each direct neighbour is 50.

The centre is therefore brighter than its neighbours by 50 intensity units.

The Laplacian response measures this local intensity difference according to the mask weights. A positive response of 200 indicates that the centre is a local bright feature relative to the four direct neighbours.

Step 6: Apply the sharpening formula

The question explicitly instructs us to add the Laplacian response to the original centre intensity.

Therefore,

I_textsharp=I_c+R

We know:

I_c=100

and

R=200

Substitute:

I_textsharp=100+200

boxedI_textsharp=300

This is the raw mathematical result before restricting the pixel to the permitted grayscale range.

Step 7: Convert the result to a valid 8-bit intensity

An 8-bit grayscale pixel must be an integer from 0 through 255.

The calculated value 300 exceeds the maximum permitted intensity.

Using clipping,

I_textvalid=min(255,max(0,I_textsharp))

Substitute 300:

I_textvalid=min(255,max(0,300))

First,

max(0,300)=300

Next,

min(255,300)=255

Therefore,

boxedI_textvalid=255

The raw sharpened value is 300, while the stored 8-bit value after clipping is 255.

Step 8: Write the final result

The Laplacian response is

boxedR=200

The raw sharpened centre intensity is

boxedI_textsharp=300

After clipping to the 8-bit grayscale range,

boxedI_textvalid=255

Important theory: Why does the sign of the mask matter?

The given mask has a positive centre coefficient and negative direct-neighbour coefficients. For this mask, the question specifies addition:

I_textsharp=I+R

Other Laplacian conventions use the opposite sign, with a negative centre coefficient and positive neighbour coefficients. Under that convention, the usual sharpening formula is subtraction:

I_textsharp=I-R

These are alternative sign conventions. The mask and sharpening formula must be consistent.

Exam conclusion: The Laplacian filter highlights local intensity changes by comparing a pixel with its neighbours. For the given image and mask, the centre response is 200. Adding this to the original centre intensity gives 300, which becomes 255 after clipping to the valid 8-bit range.

Q4. Global Thresholding and Binary Image Conversion

1. Introduction and theory

Thresholding is one of the simplest and most widely used techniques in digital image processing for separating an object from its background.

A grayscale image contains different intensity values. Thresholding converts these intensity values into two classes, usually represented by 0 and 1.

- 0 represents the background in the binary image.
- 1 represents the foreground or selected object.

The meaning of foreground and background depends on the application. For example, a document-scanning system may use thresholding to separate dark text from a light page.

What is global thresholding?

Global thresholding uses one fixed threshold value for the entire image.

If the threshold is T=100, every pixel is examined using the same rule, regardless of its position in the image.

The mathematical definition is

g(x,y)= begincases 1,& f(x,y)≥ T 0,& f(x,y)<T endcases

Here:

- f(x,y) = original pixel intensity at coordinates (x,y).
- T = selected threshold value.
- g(x,y) = output binary pixel.

The symbol ≥ means greater than or equal to. Therefore, a pixel with intensity exactly 100 must also be assigned 1.

2. Given problem

The grayscale image is

I= 40&60&120 80&150&200 30&100&180

The threshold is

boxedT=100

We must:

16. Examine every pixel.
17. Apply the threshold rule.
18. Construct the binary image.
19. Count foreground pixels.
20. Count background pixels.
21. Explain how thresholding separates objects and identify one limitation.

Step 1: Write the threshold rule

For this problem:

$g(x,y)=$ begincases 1,& $f(x,y)\geq 100$ 0,& $f(x,y)<100$ endcases

This gives us two possible outcomes.

Case 2: If the original intensity is below 100, assign 0.

Examples:

$40\rightarrow 0$

$60\rightarrow 0$

$80\rightarrow 0$

Remember that exactly 100 belongs to the first case.

Step 2: Examine the first row

The first row is

[40, 60, 120]

First pixel: 40

Compare with the threshold:

$40<100$

Therefore,

$40\rightarrow 0$

Second pixel: 60

Compare:

$60<100$

Therefore,

$60 \rightarrow 0$

Third pixel: 120

Compare:

$120 \geq 100$

Therefore,

$120 \rightarrow 1$

The first row of the binary image is

$$\boxed{[0, 0, 1]}$$

Step 3: Examine the second row

The second row is

[80, 150, 200]

First pixel: 80

$80<100$

Therefore,

$80 \rightarrow 0$

Second pixel: 150

$150 \geq 100$

Therefore,

$150 \rightarrow 1$

Third pixel: 200

$200 \geq 100$

Therefore,

$200 \rightarrow 1$

The second row becomes

boxed[0, 1, 1]

Step 4: Examine the third row

The third row is

[30, 100, 180]

First pixel: 30

$30<100$

Therefore,

$30 \rightarrow 0$

Second pixel: 100

Compare the value with the threshold:

$100 \geq 100$

Equality is allowed by the rule. Therefore,

$100 \rightarrow 1$

Third pixel: 180

$180 \geq 100$

Therefore,

$180 \rightarrow 1$

The third row becomes

boxed[0, 1, 1]

Step 5: Construct the complete binary image

Combine the three transformed rows.

First row:

[0, 0, 1]

Second row:

[0, 1, 1]

Third row:

[0, 1, 1]

The resulting binary image is

boxed B= 0&0&1 0&1&1 0&1&1

This is the required binary image.

Step 6: Construct a pixel-by-pixel threshold table

Position	Original intensity	Comparison	Binary value
Row 1, Column 1	40	40<100	0
Row 1, Column 2	60	60<100	0
Row 1, Column 3	120	120≥100	1
Row 2, Column 1	80	80<100	0
Row 2, Column 2	150	150≥100	1
Row 2, Column 3	200	200≥100	1
Row 3, Column 1	30	30<100	0
Row 3, Column 2	100	100≥100	1
Row 3, Column 3	180	180≥100	1

This table is useful in an exam because it proves that every pixel was processed using the stated rule.

Step 7: Calculate the number of foreground pixels

In the resulting binary image,

B= 0&0&1 0&1&1 0&1&1

Foreground pixels are represented by 1.

Count the ones row by row.

First row:

0+0+1=1

Second row:

0+1+1=2

Third row:

0+1+1=2

Add the row totals:

N_F=1+2+2

boxedN_F=5

Therefore, the number of foreground pixels is five.

Step 8: Calculate the number of background pixels

Background pixels are represented by 0.

First row contains two zeros.

Second row contains one zero.

Third row contains one zero.

Therefore,

N_B=2+1+1

boxedN_B=4

Alternatively, the total number of pixels is

N=3×3=9

Since five are foreground pixels,

N_B=N-N_F

N_B=9-5

boxedN_B=4

Both counting methods produce the same result.

Step 9: Explain how thresholding separates objects from the background

Thresholding separates pixels according to their intensity values.

For this problem:

- Intensities 120, 150, 200, 100, and 180 become foreground pixels.
- Intensities 40, 60, 80, and 30 become background pixels.

The output no longer preserves the original grayscale intensities. It records only which side of the threshold each pixel falls on. For instance, intensities 120 and 200 both become 1, even though their original intensities are different.

This makes thresholding useful for tasks such as:

- Document text extraction.
- Basic object detection.
- Separating bright objects from dark backgrounds.
- Preparing images for further segmentation and analysis.

Step 10: Identify one limitation

A major limitation is uneven illumination. Imagine an object photographed under a bright light on one side and a shadow on the other. Pixels belonging to the same object may have different intensities because of the lighting.

A single threshold might classify the bright portion as foreground but incorrectly classify the shadowed portion as background. Other limitations include overlapping foreground and background intensity distributions and sensitivity to noise. Adaptive thresholding can help in some cases because it calculates thresholds from local neighbourhoods instead of relying on one value for the entire image.

Exam conclusion

Global thresholding converts a grayscale image into a binary image by comparing each pixel against a fixed threshold. With T=100, the binary image is

beginbmatrix 0&0&1 0&1&1 0&1&1

The foreground count is 5 and the background count is 4. Its major limitation is that one fixed threshold may not work well under uneven lighting or when foreground and background intensities overlap.

Q5. Sobel Operator for Edge Detection

1. Introduction and theory

Edge detection is a fundamental technique in computer vision that identifies locations where image intensity changes significantly.

An edge often corresponds to a boundary between two objects or between an object and its background.

For example, when a dark object lies against a bright background, the pixels near the boundary experience a rapid intensity change. Edge detection identifies these changes.

The Sobel operator is a gradient-based edge detection method. It uses two 3×3 masks to estimate intensity changes in two perpendicular directions:

- G_x: Measures the horizontal component of the intensity gradient.
- G_y: Measures the vertical component of the intensity gradient.

These gradient components are combined to calculate the gradient magnitude.

The magnitude indicates the strength of the local intensity change.

2. Given problem

The image is

I= 10&10&10 10&50&90 10&50&90

The horizontal gradient mask is

G_x= -1&0&1 -2&0&2 -1&0&1

The vertical gradient mask is

G_y= -1&-2&-1 0&0&0 1&2&1

We must calculate G_x, G_y, the gradient magnitude, and the dominant edge direction.

Step 1: Understand the G_x mask

The mask is

G_x= -1&0&1 -2&0&2 -1&0&1

The left side contains negative weights, the middle column contains zeros, and the right side contains positive weights.

The mask compares intensities on the left and right sides of the neighbourhood.

A substantial difference between these sides produces a large horizontal gradient component. This is commonly associated with a vertical edge.

Step 2: Multiply the first row using G_x

Image first row:

[10, 10, 10]

Mask first row:

[-1, 0, 1]

Multiply corresponding values:

(-1×10)+(0×10)+(1×10)

Calculate the products:

-10+0+10

Add them:

boxedR_1=0

The first row contributes zero to G_x.

Step 3: Multiply the second row using G_x

Image second row:

[10, 50, 90]

Mask second row:

[-2, 0, 2]

Multiply corresponding values:

(-2×10)+(0×50)+(2×90)

Calculate the products:

-20+0+180

Add them:

boxedR_2=160

The second row contributes 160 to G_x.

Step 4: Multiply the third row using G_x

Image third row:

[10, 50, 90]

Mask third row:

[-1, 0, 1]

Multiply:

(-1×10)+(0×50)+(1×90)

Calculate:

-10+0+90

Add:

boxedR_3=80

The third row contributes 80 to G_x.

Step 5: Calculate the complete horizontal gradient

Add all three row contributions:

G_x=R_1+R_2+R_3

Substitute the values:

G_x=0+160+80

First:

0+160=160

Then:

160+80=240

Therefore,

boxedG_x=240

This positive value indicates that the local intensity increases in the positive horizontal direction under the specified mask convention.

Step 6: Understand the G_y mask

The vertical gradient mask is

G_y= -1&-2&-1 0&0&0 1&2&1

The top row has negative weights, the middle row contains zeros, and the bottom row has positive weights.

The mask compares intensities in the top and bottom parts of the neighbourhood.

A substantial difference between these rows produces a large vertical gradient component. This is commonly associated with a horizontal edge.

Step 7: Multiply the first row using G_y

Image first row:

[10, 10, 10]

Mask first row:

[-1, -2, -1]

Multiply corresponding values:

(-1×10)+(-2×10)+(-1×10)

Calculate:

-10-20-10

Add:

boxedR_1=-40

The first row contributes −40 to G_y.

Step 8: Multiply the second row using G_y

Image second row:

[10, 50, 90]

Mask second row:

[0, 0, 0]

Multiply:

(0×10)+(0×50)+(0×90)

=0+0+0

Therefore,

boxedR_2=0

The second row contributes zero because every mask value in that row is zero.

Step 9: Multiply the third row using G_y

Image third row:

[10, 50, 90]

Mask third row:

[1, 2, 1]

Multiply corresponding values:

(1×10)+(2×50)+(1×90)

Calculate:

10+100+90

Add the values:

boxedR_3=200

The third row contributes 200 to G_y.

Step 10: Calculate the complete vertical gradient

Add the three row contributions:

G_y=R_1+R_2+R_3

Substitute:

G_y=-40+0+200

Simplify:

G_y=-40+200

Therefore,

boxedG_y=160

The positive vertical gradient indicates an increase in intensity towards the bottom of the neighbourhood under the specified mask convention.

Step 11: Calculate the gradient magnitude

The gradient magnitude combines the two gradient components into one measure of edge strength.

Formula:

boxedG=√G_x^2+G_y^2

We have calculated:

G_x=240

G_y=160

Substitute:

G=√240^2+160^2

First, square G_x

240^2=240×240

boxed240^2=57600

Next, square G_y

160^2=160×160

boxed160^2=25600

Add the squared values

G=√57600+25600

G=√83200

Calculate the square root

sqrt83200≈288.444

Therefore, to two decimal places,

boxedG≈288.44

This is the gradient magnitude at the centre of the image.

Step 12: Determine the dominant edge direction

Compare the magnitudes of the two components:

|G_x|=240

|G_y|=160

Since

240>160

the horizontal gradient component is stronger than the vertical gradient component.

The image exhibits a stronger intensity change from left to right. Such a change is typically associated with a vertical edge.

This distinction is important:

- Gradient direction describes the direction of the strongest intensity change.
- Edge direction is perpendicular to the gradient direction.

They are not the same thing.

Step 13: Calculate the gradient orientation

The gradient orientation can be calculated using

theta=tan^-1left((G_y) ÷ (G_x)right)

Substitute the calculated components:

theta=tan^-1left((160) ÷ (240)right)

Simplify the fraction:

frac160240=(2) ÷ (3)

Therefore,

theta=tan^-1left(frac23right)

Using a calculator in degree mode:

boxedθ≈33.69^circ

This is the approximate gradient orientation measured from the positive horizontal axis for the given signed components.

The corresponding edge orientation is perpendicular to the gradient:

theta_textedge=33.69^circ+90^circ

boxedθ_textedge≈123.69^circ

Because edge orientation is commonly considered modulo 180^circ, this is equivalent to

boxedθ_textedge≈-56.31^circ

The important qualitative result is that the dominant intensity transition is horizontal, indicating a predominantly vertical edge.

Step 14: Explain the application in boundary detection and segmentation

Sobel edge detection can help a computer vision system identify where one region ends and another begins.

For example, consider an image containing a dark road and a brighter vehicle. A strong intensity transition near the vehicle boundary may produce a large gradient magnitude.

The system can use the gradient magnitude to identify candidate edge pixels. These edges can then support later tasks such as:

- Object boundary detection.
- Image segmentation.
- Shape analysis.
- Feature extraction.
- Vehicle detection and scene understanding.

However, a strong gradient does not always correspond to an object boundary. Texture, shadows, and image noise can also produce strong responses. Practical systems often combine edge information with thresholding, smoothing, or other segmentation methods.

Exam conclusion

The Sobel operator estimates horizontal and vertical intensity gradients using two 3×3 masks. For the given image,

boxed$G_x=240, G_y=160$

The gradient magnitude is

boxed$G\approx288.44$

The horizontal gradient component is dominant, indicating a stronger left-to-right intensity transition and a predominantly vertical edge. The resulting edge information can support boundary detection and image segmentation.